

Deploy e Execução em Produção do Bot FitJourneyAI

1. Introdução

Este documento descreve o processo de deploy e o ambiente de execução em produção do bot FitJourneyAI, desenvolvido no contexto deste Trabalho de Conclusão de Curso. O foco é registrar, de forma objetiva, como a aplicação poderá ser disponibilizada em um ambiente acessível pela internet, capaz de receber mensagens do Telegram 24 horas por dia e executar as rotinas de negócio e de inteligência artificial planejadas para o projeto.

2. Objetivos do Deploy e Ambiente de Execução

O objetivo principal do deploy do FitJourneyAI é colocar o bot em funcionamento em um ambiente real, permitindo que usuários interajam com o sistema por meio do Telegram, com respostas geradas pelo backend e pela API da OpenAI. Esse ambiente deve ser capaz de manter o backend em execução contínua, armazenar os dados de evolução física no banco de dados e suportar as rotinas agendadas de monitoramento de inatividade.

Neste TCC, a execução em produção é concebida para ser simples, funcional e economicamente viável, sem o objetivo de atender a cenários de alta escala, mas sim de demonstrar que o sistema é capaz de operar de forma semelhante a uma aplicação real em produção.

3. Arquitetura de Execução em Produção

A arquitetura de execução em produção do FitJourneyAI é composta pelos seguintes elementos principais:

1. Bot do Telegram (Telegram Bot API)

- Recebe as mensagens dos usuários e encaminha para o backend por meio de Webhooks.
- Entrega aos usuários as respostas geradas pela aplicação.

2. Backend (aplicação Spring Boot)

- Implementa os fluxos de interação: onboarding, registro de dados, geração de treinos, visualização de progresso, resumos e reengajamento de usuários inativos.
- Expõe um endpoint HTTP/HTTPS que funciona como Webhook para o Telegram.
- Comunica-se com a API da OpenAI por meio de chamadas HTTP autenticadas.
- Acessa o banco de dados PostgreSQL para leitura e gravação de informações.
- Executa rotinas agendadas com `@Scheduled`, como o monitoramento de inatividade.

3. Banco de Dados PostgreSQL

- Armazena usuários, registros de peso e medidas, treinos realizados, histórico relevante e metadados de inatividade (por exemplo, `last\_interaction\_at` e `last\_nudge\_at`).

4. API da OpenAI

- Gera treinos personalizados, comentários de progresso e mensagens motivacionais com base nos dados do usuário.

3.1 Fluxo de requisições em produção

Em produção, o fluxo básico de interação é o seguinte:

1. O usuário envia uma mensagem para o bot no Telegram.
2. A Telegram Bot API encaminha essa mensagem, em formato JSON, para a URL de Webhook configurada no backend.
3. O WebhookController recebe a requisição e a repassa para o MessageOrchestrator.
4. O MessageOrchestrator identifica o tipo de mensagem (comando, texto livre, fluxo em andamento) e aciona os serviços necessários (acesso ao banco, chamadas à OpenAI, regras de negócio).
5. A resposta é construída e enviada ao Telegram por meio do TelegramService.
6. A Telegram Bot API entrega a resposta ao usuário na interface do aplicativo.

4. Ambiente de Hospedagem

O FitJourneyAI foi planejado para rodar tanto em ambiente de desenvolvimento local quanto em um ambiente de produção acessível pela internet. A seguir são descritos esses dois contextos.

4.1 Ambiente de desenvolvimento local

Durante o desenvolvimento, a aplicação é executada localmente, utilizando:

- Spring Boot em modo de desenvolvimento (por exemplo, via IntelliJ ou `mvn spring-boot:run`);
- Uma instância local de PostgreSQL ou um container de banco de dados em Docker;
- Ferramentas como Ngrok para expor temporariamente um endpoint público (HTTPS) quando for necessário testar o Webhook do Telegram a partir da máquina local.

4.2 Ambiente de produção para este TCC

Para a execução em produção no contexto deste TCC, a solução foi pensada para ser hospedada em um servidor na nuvem capaz de manter o backend em execução contínua. Esse servidor pode ser obtido por meio de camadas gratuitas (free tier) de provedores de nuvem ou por meio de uma VPS (Virtual Private Server) de baixo custo.

A configuração prevista inclui:

- Um sistema Linux (por exemplo, Ubuntu Server) executando o backend e o banco de

dados;

- Java instalado para execução da aplicação Spring Boot;
- Docker e Docker Compose como opção para organizar os serviços em containers;
- Banco de dados PostgreSQL instalado localmente no servidor ou fornecido como serviço gerenciado.

4.3 Requisitos mínimos de hardware e software

Considerando o uso esperado durante o TCC, uma configuração mínima adequada para o servidor é:

- 1 vCPU;
- 1 a 2 GB de RAM;
- 20 a 30 GB de armazenamento em disco.

Em relação a software, são utilizados:

- Sistema operacional Linux (como Ubuntu Server);
- Java 17 ou versão compatível com o projeto;
- Docker e Docker Compose (quando o deploy é feito com containers);
- PostgreSQL;
- Opcionalmente, Nginx configurado como proxy reverso para HTTPS.

5. Estratégia de Deploy

A estratégia de deploy do FitJourneyAI foi planejada para ser simples de executar e fácil de reproduzir em outros ambientes. Neste trabalho são consideradas duas formas principais de disponibilizar o backend em produção: executando diretamente o arquivo JAR ou utilizando containers Docker.

5.1 Execução direta do arquivo JAR

Na abordagem mais direta, a aplicação Spring Boot é empacotada como um arquivo JAR executável, gerado com o comando ``mvn clean package``. O arquivo resultante (por exemplo, ``fitjourneyai.jar``) é copiado para o servidor de produção e executado com:

```
`java -jar fitjourneyai.jar`
```

No servidor Linux, é possível registrar esse comando como um serviço do sistema (por exemplo, via `systemd`), fazendo com que a aplicação seja iniciada automaticamente junto com o sistema operacional e reiniciada em caso de falhas.

5.2 Deploy com Docker e Docker Compose

Uma segunda abordagem, também considerada neste trabalho, é o deploy com Docker. Nessa opção, a aplicação Spring Boot é empacotada em uma imagem Docker, e o banco de dados PostgreSQL é executado em um container separado. Um arquivo ``docker-`

`compose.yml` pode ser utilizado para orquestrar ambos os serviços.

Com o uso de Docker Compose, o ambiente de produção pode ser iniciado com um único comando, como:

```
`docker compose up -d`
```

Essa abordagem facilita a replicação do ambiente em outras máquinas e reduz problemas de configuração, uma vez que as versões das dependências ficam definidas nos arquivos de configuração dos containers.

5.3 Variáveis de ambiente e segredos

Durante o deploy, os dados sensíveis utilizados pelo FitJourneyAI são configurados por meio de variáveis de ambiente. Entre esses dados, destacam-se:

- URL de conexão com o banco de dados (host, porta, nome do banco, usuário, senha);
- Token de acesso do bot do Telegram;
- Chave de API da OpenAI.

Essas variáveis não são armazenadas diretamente no código-fonte, evitando que segredos sejam expostos em repositórios de controle de versão. Em ambientes Docker, é possível utilizar arquivos `.env` ou configurações do próprio provedor de nuvem para definir esses valores de forma segura.

6. Configuração do Banco de Dados PostgreSQL

O PostgreSQL é o banco de dados relacional utilizado pelo FitJourneyAI. No ambiente de produção, serão criados:

- Um banco de dados específico para o projeto (por exemplo, `fitjourneyai_db`);
- Um usuário com permissões para leitura e escrita nesse banco;
- As tabelas necessárias para armazenar usuários, registros de medidas, treinos e demais informações.

Ferramentas de migração, como Flyway ou Liquibase, podem ser utilizadas para manter o schema do banco alinhado com a versão da aplicação. Dessa forma, ao iniciar o backend em produção, as migrações são aplicadas automaticamente, criando ou atualizando as estruturas necessárias.

7. Configuração do Bot no Telegram

Para que o backend receba as mensagens enviadas ao bot, é necessário configurar o bot no Telegram e associar o token gerado ao servidor em produção.

7.1 Criação do bot via BotFather

A criação do bot é feita da seguinte forma:

1. No aplicativo Telegram, é aberto um chat com o contato `@BotFather`.
2. O comando `/newbot` é enviado e o assistente solicita um nome e um username para o bot.
3. Ao final do processo, o BotFather fornece um token de API, que é utilizado pelo backend para se comunicar com a Telegram Bot API. Esse token é configurado como variável de ambiente no servidor.

7.2 Configuração do Webhook

Com a aplicação em execução em um servidor acessível via HTTPS, é necessário informar ao Telegram qual URL deve ser usada como Webhook. Isso é feito por meio de uma requisição HTTP para a API do Telegram, por exemplo:

- Endpoint: `https://api.telegram.org/bot<SEU\_TOKEN>/setWebhook`
- Parâmetro: `url=https://seu-dominio-ou-ip/webhook/telegram`

Após essa configuração, todas as mensagens enviadas pelos usuários ao bot são encaminhadas automaticamente para o endpoint configurado, onde o WebhookController do FitJourneyAI fará o tratamento.

8. HTTPS e Segurança Básica

Para o funcionamento correto do Webhook e para a proteção das comunicações, é importante que o endpoint do backend esteja exposto por meio de HTTPS. Além disso, é desejável adotar cuidados básicos de segurança na configuração do servidor.

8.1 Uso de proxy reverso (Nginx)

Uma forma comum de habilitar HTTPS é utilizar o Nginx como proxy reverso. Nesse cenário:

- O Nginx escuta na porta 443 (HTTPS) e recebe as requisições provenientes do Telegram;
- O Nginx encaminha essas requisições para a porta interna na qual o Spring Boot está rodando (por exemplo, 8080);
- O certificado TLS é instalado e gerenciado no Nginx, podendo ser obtido gratuitamente por meio do Let's Encrypt.

8.2 Proteção de tokens e chaves

Os tokens do Telegram, as chaves de API da OpenAI e as credenciais do banco de dados são informações sensíveis. Neste projeto, esses valores são mantidos fora do código-fonte e configurados por meio de variáveis de ambiente ou arquivos de configuração protegidos no servidor. O acesso a esses arquivos é restrito ao usuário responsável pela execução da aplicação.

9. Execução Contínua e Rotinas Agendadas

Para que o FitJourneyAI acompanhe a evolução dos usuários e realize ações automáticas, como o envio de lembretes em caso de inatividade, é necessário manter o backend em execução contínua e garantir o funcionamento das rotinas agendadas.

9.1 Manter o serviço sempre ativo

No servidor Linux, a aplicação pode ser registrada como serviço do sistema (no caso de execução via JAR) ou configurada com políticas de reinício automático (no caso de execução via Docker). Dessa forma, em caso de reinicialização do servidor ou falhas pontuais, o backend é automaticamente iniciado novamente.

9.2 Rotinas `@Scheduled` no Spring Boot

O uso de `@Scheduled` no Spring Boot permite que o FitJourneyAI execute tarefas de forma periódica, sem depender de interação manual. Entre as rotinas planejadas neste projeto, destacam-se:

- Verificação periódica de usuários inativos, com envio de mensagens de reengajamento após um determinado período sem interação;
- Possível geração de resumos semanais de progresso, a serem enviados automaticamente aos usuários.

Essas rotinas utilizam o mesmo ambiente de execução do backend, o que simplifica a infraestrutura, já que não é necessário configurar um serviço externo de agendamento.

10. Monitoramento e Logs

Para acompanhar o funcionamento da aplicação em produção, o FitJourneyAI utiliza logs gerados pelo próprio Spring Boot. Esses logs permitem identificar erros, analisar o comportamento do sistema e realizar ajustes quando necessário.

Entre as informações registradas nos logs, podem estar:

- Erros de comunicação com o Telegram ou com a OpenAI;
- Exceções inesperadas durante o processamento de mensagens;
- Mensagens de depuração associadas a fluxos importantes, como registro de dados ou execução de rotinas agendadas.

11. Custos e Viabilidade Econômica

Durante o desenvolvimento deste TCC foi considerada a necessidade de manter o FitJourneyAI em um ambiente online sem gerar custos elevados. Por esse motivo, a estratégia de deploy leva em conta o uso de camadas gratuitas de provedores de nuvem e recursos de baixo custo.

Com uma quantidade reduzida de usuários e uso moderado da API da OpenAI, é possível operar o sistema com recursos limitados de CPU e memória, o que permite utilizar planos

gratuitos ou VPS de entrada. O consumo da API da OpenAI também pode ser controlado, priorizando modelos mais econômicos e evitando chamadas desnecessárias.

12. Considerações Finais

Este documento apresenta a forma como o deploy e a execução em produção do FitJourneyAI foram planejados no contexto deste TCC. A combinação de um backend em Spring Boot, um banco de dados PostgreSQL, a integração com o Telegram e a API da OpenAI, somada à possibilidade de uso de servidores em nuvem de baixo custo ou free tier, mostra que a aplicação pode ser executada em um ambiente próximo ao de produção real, com viabilidade técnica e econômica.

As decisões descritas aqui reforçam que o projeto não se limita à implementação de código, mas contempla também as etapas necessárias para colocar o sistema em funcionamento de forma contínua e acessível, alinhando-se às boas práticas de desenvolvimento e deploy de aplicações modernas baseadas em serviços na nuvem.